Laporan Tugas Besar

IF2224 Teori Bahasa Formal dan Otomata

Milestone 1

Semester II / 2025–2026

Disusun oleh:

GTFOTBFO (HBB)

Nathanael Imandatua Manurung - 13524021
Nicholas Wise Saragih Sumbayak - 13524037
Kurt Mikhael Purba - 13524065
Rava Khoman Tuah Saragih - 13524107

Laboratorium Ilmu dan Rekayasa Komputasi
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Daftar Isi

1 Pendahuluan **3**

2 Dasar Teori **3**
2.1 Pendefinisian Bahasa Pemrograman . . . 3
2.1.1 Hierarki Bahasa Pemrograman . . . 4
2.1.2 *Compiler* dan *Interpreter* . . . 4
2.2 Teori Kompilator (*Compiler Theory*) . . . 4
2.2.1 Analisis Leksikal (*Lexical Analysis*) . . . 4
2.2.2 Elemen Bahasa: *Token*, *Lexeme*, dan *Pattern* . . . 5
2.3 Teori Bahasa Formal dan Otomata . . . 5
2.3.1 *Regular Expression* . . . 5
2.3.2 *Finite Automata* . . . 5

3 Implementation **6**
3.1 System Architecture . . . 6
3.2 Directory Structure . . . 6
3.3 Backend . . . 6
3.3.1 Entry Point . . . 6
3.3.2 Core Algorithm Package . . . 6
3.3.3 API Endpoints . . . 7
3.4 Frontend . . . 7
3.4.1 Component Architecture . . . 7
3.4.2 State Management . . . 7
3.4.3 UI/UX Principles Applied . . . 7
3.5 Data Layer . . . 8
3.5.1 Dataset . . . 8
3.5.2 Preprocessing Pipeline . . . 8
3.6 Containerisation with Docker . . . 8
3.6.1 Multi-stage Build Strategy . . . 8
3.6.2 Running Locally . . . 8
3.7 Performance Metrics . . . 9

4 Experiments and Results **9**
4.1 Test Case Category A . . . 9
4.1.1 Test Case 1: *[Description]* . . . 9
4.1.2 Test Case 2: *[Description]* . . . 9
4.2 Test Case Category B . . . 10
4.3 Overall Analysis . . . 10
4.3.1 Algorithm A — Strengths and Limitations . . . 10
4.3.2 Algorithm B — Strengths and Limitations . . . 10

5 Conclusion **10**
5.1 Summary . . . 10
5.2 Suggestions for Future Work . . . 10
5.3 Individual Reflections . . . 10
5.3.1 Nathanael Imandatua Manurung - 13524021 . . . 10
5.3.2 Nicholas Wise Saragih Sumbayak - 13524037 . . . 10

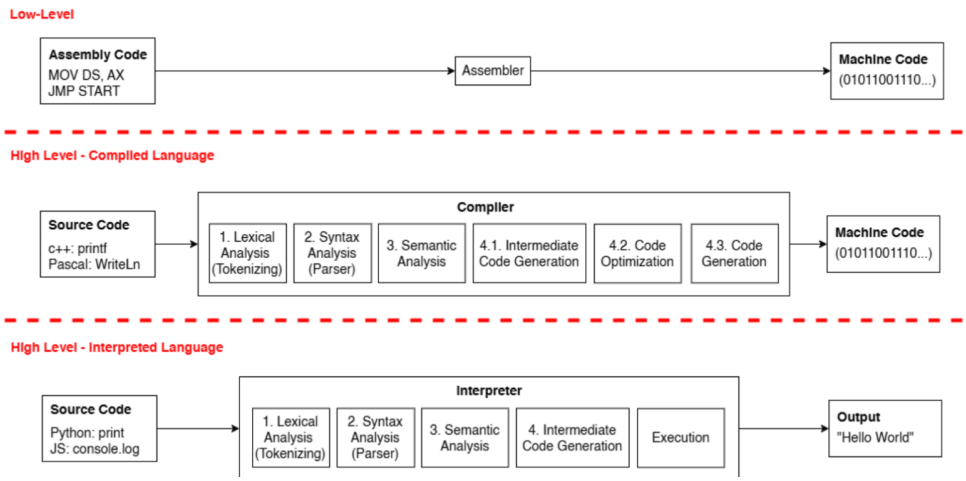
5.3.3 Kurt Mikhael Purba - 13524065 11

References **12**

Appendix **13**

1 Pendahuluan

Bahasa Pemrograman diciptakan sebagai perantara komunikasi antara manusia yang mengerti logika dan bahasa tingkat tinggi dengan komputer yang hanya dapat mengerti bahasa mesin (*machine code*). Tanpa adanya bahasa pemrograman, manusia harus menuliskan instruksi komputer secara tingkat rendah kepada komputer yang hanya mengerti 0 dan 1. Dalam hierarkinya, bahasa tingkat tinggi (*high-level*) dirancang agar lebih mendekati bahasa manusia, namun hal ini menuntut proses transformasi yang berlapis agar dapat dipahami oleh CPU.



Gambar 1: Tipe-tipe bahasa pemrograman

Sumber: Asisten IF2224

Proses transformasi ini dilakukan melalui mekanisme *compiler* atau *interpreter*. Tahap awal dan fundamental dalam mekanisme tersebut adalah Analisis Leksikal (*Lexical Analysis*). Pada tahap ini, kode sumber yang berupa rangkaian karakter mentah dibaca dan dikelompokkan menjadi satuan makna terkecil yang disebut token, seperti kata kunci (*keyword*), pengenal (*identifier*), operator, dan konstanta. Analisis leksikal berperan krusial dalam menyederhanakan tugas tahap berikutnya, yaitu parsing, dengan membuang elemen yang tidak relevan seperti spasi dan komentar, serta mendeteksi simbol invalid sejak awal.

2 Dasar Teori

2.1 Pendefinisian Bahasa Pemrograman

Bahasa Pemrograman adalah cara manusia untuk dapat memberikan perintah (*instruction*) kepada sebuah komputer. Setiap bahasa pemrograman memiliki kata kunci dan aturannya masing-masing serta makna dari kata-kata itu yang disebut sintaks(*syntax*) dan semantik (*semantics*). Input yang terdapat kesalahan pada tingkat sintaks berarti input tersebut tidak valid untuk bahasa pemrograman tersebut, sedangkan kesalahan pada tingkat semantik umumnya berbentuk kesalahan logika.

2.1.1 Hierarki Bahasa Pemrograman

Berdasarkan tingkat abstraksinya, bahasa pemrograman diklasifikasikan menjadi dua kategori utama:

- ***Low-Level Language* (Bahasa Tingkat Rendah)**: Bahasa yang instruksinya mendekati arsitektur perangkat keras dan bahasa mesin (biner 0 dan 1). Contohnya adalah *Assembly*, yang memerlukan *Assembler* untuk menerjemahkan kode menjadi instruksi yang dapat dipahami CPU.
- ***High-Level Language* (Bahasa Tingkat Tinggi)**: Bahasa yang dirancang agar lebih mudah dibaca dan dipahami oleh manusia karena menggunakan kosakata yang menyerupai bahasa natural. Semakin tinggi tingkat bahasa tersebut, semakin banyak tahapan pemrosesan yang diperlukan untuk mengubahnya menjadi kode mesin. Contoh bahasa tingkat tinggi antara lain Pascal, C++, Python, dan Java.

2.1.2 *Compiler* dan *Interpreter*

Untuk menjalankan kode sumber yang ditulis dalam bahasa tingkat tinggi, diperlukan sebuah perangkat lunak penerjemah yang mengubah kode tersebut menjadi format yang dapat dieksekusi oleh mesin. Secara umum, terdapat dua mekanisme utama untuk mengubah bahasa tingkat tinggi ke bahasa mesin:

1. ***Compiler* (Kompilator)**: *Compiler* adalah program yang menerjemahkan seluruh *source code* secara utuh menjadi kode mesin (biasanya dalam bentuk *Object File*) sebelum program dijalankan. Proses kompilasi pada *compiler* umumnya terdiri dari empat tahapan utama: *Lexical Analysis*, *Syntax Analysis*, *Semantic Analysis*, dan *Intermediate Code Generation*. Karakteristik utama *compiler* adalah jika terdeteksi adanya kesalahan (*error*) di dalam *source code*, maka seluruh proses kompilasi akan gagal dan program tidak dapat dijalankan.
2. ***Interpreter***: Berbeda dengan *compiler*, *interpreter* memproses kode sumber baris demi baris secara langsung. *Interpreter* membaca, menerjemahkan, dan langsung mengeksekusi kode mesin tersebut baris demi baris. Hal ini memungkinkan program tetap berjalan normal hingga baris sebelum ditemukannya kesalahan; misalnya, jika terdapat *error* pada baris ke-10, baris 1 sampai 9 akan tetap berhasil dieksekusi. Meskipun mekanisme eksekusinya berbeda, *interpreter* sebenarnya memiliki tahapan pemrosesan internal yang serupa dengan *compiler*.

2.2 Teori Kompilator (*Compiler Theory*)

Kompilator (*compiler*) bekerja melalui beberapa tahapan yang berurutan. Tahapan paling awal dan krusial yang berhubungan langsung dengan pembacaan kode sumber adalah Analisis Leksikal.

2.2.1 Analisis Leksikal (*Lexical Analysis*)

Analisis leksikal adalah fase pertama dalam proses kompilasi. Komponen yang melakukan tugas ini disebut *lexical analyzer* atau *scanner*. Tugas utamanya adalah membaca barisan karakter dalam kode sumber (*source code*) dari kiri ke kanan dan mengelompokkannya ke dalam unit-unit logis yang bermakna, yang disebut *token*. Selain menghasilkan *token*, tahap ini juga memiliki beberapa peran pendukung:

- **Menghapus karakter yang tidak bermakna**: *Lexer* akan mengabaikan spasi kosong (*whitespace*), *tab*, dan baris baru (*newline*) yang tidak mempengaruhi logika program.
- **Membuang komentar**: Teks yang ditulis sebagai komentar untuk dokumentasi juga akan dihapus agar tidak ikut diproses oleh tahap selanjutnya (*parser*).
- **Penanganan Kesalahan (*Error Handling*)**: Melacak nomor baris kode sumber untuk memberikan laporan lokasi kesalahan (seperti *lexical error*) apabila ditemukan urutan karakter yang tidak mengikuti aturan bahasa pemrograman.

2.2.2 Elemen Bahasa: *Token*, *Lexeme*, dan *Pattern*

Dalam melakukan analisis leksikal, terdapat tiga elemen bahasa utama yang harus dipahami secara independen:

- ***Token***: Merupakan entitas logis yang merepresentasikan nama dari sebuah kategori, seperti *identifier*, *keyword*, operator (*plus*, *minus*), atau angka (*intcon*). Ini adalah format akhir yang akan diberikan oleh *lexer* kepada *parser*.
- ***Lexeme***: Urutan karakter nyata (aktual) yang terdapat di dalam kode sumber yang berhasil dicocokkan dengan suatu pola pembentuk *token*. Contohnya, jika program membaca kata `Mylnt`, maka `Mylnt` adalah *lexeme*-nya, sedangkan tipe *token*-nya adalah *ident*.
- ***Pattern* (Pola)**: Aturan atau deskripsi formal yang mendefinisikan himpunan karakter apa saja yang diizinkan untuk membentuk sebuah *token* tertentu.

2.3 Teori Bahasa Formal dan Otomata

2.3.1 *Regular Expression*

Regular Expression (RE) adalah sebuah notasi matematis formal yang digunakan untuk mendeskripsikan suatu bahasa reguler. Dalam kompilator, RE digunakan secara luas untuk mendefinisikan *pattern* (pola) dari *token* secara akurat. Dengan menggunakan operasi dasar seperti *Concatenation* (penyambungan), *Union* (penggabungan alternatif), dan *Kleene Closure* (perulangan nol kali atau lebih), pembuat bahasa dapat mendefinisikan aturan rumit, misalnya: "Sebuah identifier harus diawali dengan huruf, diikuti oleh kombinasi huruf atau angka dengan jumlah berapa pun."

2.3.2 *Finite Automata*

Finite State Machines (FSM) atau *Finite Automata* (FA) adalah model mesin komputasi abstrak yang digunakan untuk mengenali *regular expression*. Jika *regular expression* mendefinisikan aturannya, maka *finite automata* adalah mesin eksekutor yang memvalidasi apakah sebuah *lexeme* cocok dengan aturan tersebut.

Sebuah FA direpresentasikan melalui lima elemen utama (*tuple*): himpunan *state* yang terbatas, himpunan simbol *input* (alfabet), aturan/fungsi transisi, satu *state* awal (*initial state*), dan himpunan *state* akhir/penerima (*accepting states*). Berdasarkan sifat determinasinya, *Finite Automata* dibagi menjadi dua jenis:

1. ***Nondeterministic Finite Automata* (NFA)**: Suatu mesin di mana sebuah *state* dapat memiliki nol, satu, atau lebih transisi untuk satu simbol *input* yang sama, termasuk transisi hampa (*epsilon transition*).

2. ***Deterministic Finite Automata* (DFA)**: Suatu mesin di mana setiap *state* hanya memiliki tepat satu transisi untuk setiap simbol *input*.

Dalam pembuatan *lexical analyzer*, model DFA jauh lebih sering digunakan atau diimplementasikan secara terprogram ke dalam kode komputer (seperti C/C++) karena sifat penentuan jalurnya yang deterministik, mutlak, dan tidak ambigu untuk setiap karakter yang dibaca.

3 Implementation

3.1 System Architecture

The application follows a **client-server** architecture with a clear separation between the presentation layer (frontend) and the application/data layer (backend).

3.2 Directory Structure

Listing 1: Project directory layout

```
project-root/
    src/
        backend/          # Server-side code
            main.go
            algeo/        # Core algorithm package
            preprocess/   # Data preprocessing
            caching/      # Model persistence
        frontend/         # Client-side code
            src/
                components/
                pages/
                hooks/
    data/
        covers/           # Image assets
        txt/              # Text corpus
        mapper.json       # Metadata index
    be.Dockerfile
    fe.Dockerfile
```

3.3 Backend

3.3.1 Entry Point

3.3.2 Core Algorithm Package

Listing 2: Core algorithm — pseudocode / excerpt

Algorithm

```go
func ComputeSomething(input Matrix, k int) (*Result, error) {
    // Step 1: preprocess
    // Step 2: core computation
    // Step 3: return result
}
```

Algorithm steps:

1. Step one description.
2. Step two description.
3. Step three description.

Algorithm B ...

3.3.3 API Endpoints

Method	Endpoint	Input	Description
GET	`/items`	query params	List items with pagination
GET	`/items/:id`	path param	Item detail + recommendations
POST	`/search/image`	multipart	Visual similarity search
POST	`/search/text`	JSON body	Semantic text search

Tabel 1: REST API endpoints

3.4 Frontend

3.4.1 Component Architecture

The frontend is a Single Page Application (SPA) built with React + TypeScript. Key design choices:

- **React Router** — client-side routing with no full page reloads.
- **Context API** — global state management without external libraries.
- **Tailwind CSS** — utility-first styling for responsive layouts.

3.4.2 State Management

3.4.3 UI/UX Principles Applied

1. **Visual Hierarchy** — consistent typographic scale across all views.
2. **Responsive Design** — mobile-first breakpoints (`sm`, `md`, `lg`).
3. **Micro-interactions** — subtle animations via Framer Motion for tactile feedback.
4. **Error & Empty States** — informative messages when searches return no results.

3.5 Data Layer

3.5.1 Dataset

The dataset consists of **N** items stored as file-based records:

```
data/
    mapper.json          # ID -> metadata mapping
    covers/              # Image files (.jpg)
    txt/                 # Text content (.txt)
```

3.5.2 Preprocessing Pipeline

1. **Load** raw files from disk.
2. **Normalise** (grayscale conversion, lowercasing, tokenisation, etc.).
3. **Index** into an efficient in-memory structure.
4. **Cache** computed representations to disk via `encoding/gob` (or pickle, etc.).

3.6 Containerisation with Docker

3.6.1 Multi-stage Build Strategy

Both backend and frontend use multi-stage Dockerfiles to minimise final image size.

Listing 3: Backend Dockerfile (abbreviated)

```
# Stage 1      compile
FROM golang:1.22 AS builder
WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN GOOS=linux go build -o server ./main.go

# Stage 2      minimal runtime
FROM ubuntu:22.04
WORKDIR /app
COPY --from=builder /app/server .
VOLUME ["/app/data"]
EXPOSE 8080
ENTRYPOINT ["./server"]
```

3.6.2 Running Locally

```
# Backend
docker build -t myapp-backend -f be.Dockerfile .
docker run --rm -p 8080:8080 \
  -e FRONTEND_URL=http://localhost:3000 \
  -v "$(pwd)/data:/app/data:ro" \
  myapp-backend
```

```

# Frontend
docker build -t myapp-frontend \
  --build-arg VITE_API_URL=http://localhost:8080 \
  -f fe.Dockerfile .
docker run --rm -p 3000:80 myapp-frontend
```

3.7 Performance Metrics

Operation	Time
Document preprocessing	7.67 s
Vocabulary build	1.24 s
Term-document matrix	477 ms
SVD (50 components)	6.06 s
Model training (total)	23.0 s
Query response (P50)	< 50 ms
Query response (P99)	< 100 ms

Tabel 2: Training and inference performance (N = 523 documents)

4 Experiments and Results

4.1 Test Case Category A

4.1.1 Test Case 1: *[Description]*

Input: Describe the input used.
Expected output: What the correct result should be.

[Screenshot placeholder]

Gambar 2: Test case 1 — result screenshot

Analysis: Explain whether the result is correct and why.

4.1.2 Test Case 2: *[Description]*

...

4.2 Test Case Category B

...

4.3 Overall Analysis

4.3.1 Algorithm A — Strengths and Limitations

Strengths:

- Strength one.
- Strength two.

Limitations:

- Limitation one.
- Limitation two.

4.3.2 Algorithm B — Strengths and Limitations

...

5 Conclusion

5.1 Summary

1. **Finding 1** — description.
2. **Finding 2** — description.
3. **Finding 3** — description.

5.2 Suggestions for Future Work

1. Idea for improvement or extension one.
2. Idea for improvement or extension two.

5.3 Individual Reflections

5.3.1 Nathanael Imandatua Manurung - 13524021

Role:
Reflection: Write your personal reflection here.

5.3.2 Nicholas Wise Saragih Sumbayak - 13524037

Role:
Reflection: ...

5.3.3 Kurt Mikhael Purba - 13524065

Role:
Reflection: ...

References

[1] A. Author, *Book Title*, Edition. City: Publisher, Year.

[2] A. Author and B. Author, "Article title," *Journal Name*, vol. X, no. Y, pp. 1–10, Month Year. https://doi.org/10.xxxx/xxxxx.

[3] Organisation, "Web resource title," Year. [Online]. Available: https://example.com.

Appendix

Resource	Link
GitHub Repository	https://github.com/your-org/your-repo
Live Deployment	https://your-deployment-url.example.com

Tabel 3: Repository and deployment links